

**LAPORAN TUGAS SISTEM OPERASI
DEADLOCK**



**Dosen Pengampu :
Triawan Adi Cahyanto M.Kom**

**Disusun Oleh :
Subhan Maulana Andrianto 2410651014**

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025**

Tugas Praktikum

Tugas 1: Analisis Deadlock

Jalankan program `deadlock_simple.c` dan jelaskan:

1. Mengapa program mengalami deadlock?
2. Kondisi deadlock apa saja yang terpenuhi?
3. Gambar diagram resource allocation graph-nya

Tugas 2: Implementasi Pencegahan

Modifikasi program `deadlock_simple.c` menggunakan teknik:

1. Lock ordering
2. Try-lock dengan retry
3. Bandingkan kedua metode tersebut

Tugas 3: Banker's Algorithm

Buat program baru dengan data :

- 4 proses
- 3 jenis resource
- Available: [5, 4, 3]
- Implementasikan request resource dan cek keamanan sistem

Tugas 4: Kasus Nyata (Real Case)

Buat simulasi deadlock untuk kasus: “Dua orang ingin transfer uang antar rekening secara bersamaan”

- Orang A: transfer dari rekening 1 ke rekening 2
- Orang B: transfer dari rekening 2 ke rekening 1
- Implementasikan deadlock prevention

Jawaban

(ANALISIS DEADLOCK)

1. Mengapa program mengalami deadlock?

Deadlock terjadi karena:

- Thread 1 memegang lock1, lalu menunggu lock2
- Thread 2 memegang lock2, lalu menunggu lock1

Tidak ada thread yang dapat melanjutkan sehingga program hang selamanya.

2. Kondisi Deadlock (Coffman Conditions) yang terpenuhi

Deadlock terjadi karena semua kondisi berikut terpenuhi:

1. Mutual Exclusion

- Lock bersifat eksklusif: hanya 1 thread dapat memegang 1 lock dalam satu waktu.

2. Hold and Wait

- Thread 1 memegang lock1 sambil menunggu lock2
- Thread 2 memegang lock2 sambil menunggu lock1

3. No Preemption

- Lock tidak dapat direbut paksa oleh thread lain.

4. Circular Wait

- T1 menunggu lock milik T2
- T2 menunggu lock milik T1
- Terjadi siklus.

3. T1 → meminta Lock2

↑ ↓

Lock1 ← T2 ← Lock2

↑ ↓

meminta Lock1

Siklus:

T1 → Lock2 → T2 → Lock1 → T1

Siklus ini menandakan deadlock.

(IMPLEMENTASI PENCEGAHAN)

1. Teknik lock ordering

Solusi: Semua thread HARUS mengambil lock dalam urutan yang sama, yaitu:
Lock 1 → Lock 2

```
#include <stdio.h>

#include <pthread.h>

#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

// Fungsi helper: ambil lock sesuai urutan global
void acquire_in_order() {
    printf(" → Mengambil Lock 1...\n");
    pthread_mutex_lock(&lock1);

    printf(" → Mengambil Lock 2...\n");
    pthread_mutex_lock(&lock2);
}

void release_in_order() {
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    printf(" → Semua lock dilepas\n");
}

void* thread1_func(void* arg) {
    printf("Thread 1 mulai.\n");

    acquire_in_order();
```

```

    printf("Thread 1: Bekerja ...\n");
    sleep(1);
    release_in_order();

    return NULL;
}

void* thread2_func(void* arg) {
    printf("Thread 2 mulai.\n");

    acquire_in_order();
    printf("Thread 2: Bekerja ...\n");
    sleep(1);
    release_in_order();

    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== PENCEGAHAN DEADLOCK: LOCK ORDERING ===\n\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\nProgram selesai tanpa deadlock.\n");
}

```

```
return 0;
```

```
}
```

2. VERSI TRY-LOCK + RETRY

Solusi: Jika mencoba lock kedua tapi gagal → lepaskan lock pertama → retry.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void* thread1_func(void* arg) {
    while (1) {
        printf("Thread 1: Mencoba Lock 1...\n");
        pthread_mutex_lock(&lock1);

        printf("Thread 1: Mencoba Lock 2 (try)...\n");
        if (pthread_mutex_trylock(&lock2) == 0) {
            printf("Thread 1: Dapat kedua lock ✓\n");
            sleep(1);

            pthread_mutex_unlock(&lock2);
            pthread_mutex_unlock(&lock1);
            break;
        } else {
            printf("Thread 1: Gagal Lock 2 → Melepas Lock 1 dan retry...\n");
            pthread_mutex_unlock(&lock1);
            usleep(100000); // 0.1 detik
        }
    }
}
```

```

    printf("Thread 1 selesai.\n");
    return NULL;
}

void* thread2_func(void* arg) {
    while (1) {
        printf("Thread 2: Mencoba Lock 2...\n");
        pthread_mutex_lock(&lock2);

        printf("Thread 2: Mencoba Lock 1 (try)...\n");
        if (pthread_mutex_trylock(&lock1) == 0) {
            printf("Thread 2: Dapat kedua lock ✓\n");
            sleep(1);

            pthread_mutex_unlock(&lock1);
            pthread_mutex_unlock(&lock2);
            break;
        } else {
            printf("Thread 2: Gagal Lock 1 → Melepas Lock 2 dan retry...\n");
            pthread_mutex_unlock(&lock2);
            usleep(150000); // 0.15 detik
        }
    }
}

printf("Thread 2 selesai.\n");
return NULL;
}

int main() {
    pthread_t t1, t2;

```



```
printf("=== PENCEGAHAN DEADLOCK: TRY-LOCK + RETRY ===\n\n");

pthread_create(&t1, NULL, thread1_func, NULL);
pthread_create(&t2, NULL, thread2_func, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

printf("\nProgram selesai tanpa deadlock.\n");
return 0;
}
```

3. PERBANDINGAN KEDUA METODE

Metode	Cara Kerja	Keunggulan	Kekurangan	Cocok Untuk
Lock Ordering	Semua thread mengambil lock dalam urutan sama	Sederhana, cepat, efektif	Butuh urutan global, tidak fleksibel	Program dengan resource tetap
Try-lock + Retry	Jika gagal lock kedua → lepas lock pertama → retry	Fleksibel, mencegah hold-and-wait	Banyak retry, CPU bisa naik	Sistem dinamis / banyak resource

(BANKER'S ALGORITHM)

```
#include <stdio.h>

#define P 4 // jumlah proses
#define R 3 // jumlah resource

int available[R] = {5, 4, 3}; // Available awal
int max[P][R];           // Matriks Maksimum
int alloc[P][R];         // Matriks Alokasi
int need[P][R];          // Matriks Need

// Mengecek apakah sistem dalam keadaan aman
int isSafe() {
    int work[R], finish[P] = {0};
    for (int i = 0; i < R; i++)
        work[i] = available[i];

    int count = 0;

    while (count < P) {
        int found = 0;

        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                int canAllocate = 1;
                for (int j = 0; j < R; j++) {
                    if (need[i][j] > work[j]) {
                        canAllocate = 0;
                        break;
                    }
                }
            }
        }
    }
}
```

```

    }

    if (canAllocate) {
        for (int j = 0; j < R; j++)
            work[j] += alloc[i][j];

        finish[i] = 1;
        found = 1;
        count++;
    }
}

if (!found)
    return 0; // Tidak aman
}

return 1; // Aman
}

// Fungsi request resource
int requestResource(int p, int req[]) {
    // Cek apakah request <= need
    for (int j = 0; j < R; j++) {
        if (req[j] > need[p][j]) {
            printf("Error: Request melebihi need!\n");
            return 0;
        }
    }
}

```

```

// Cek apakah request <= available
for (int j = 0; j < R; j++) {
    if (req[j] > available[j]) {
        printf("Resource tidak cukup. Proses harus menunggu.\n");
        return 0;
    }
}

// Coba alokasikan sementara
for (int j = 0; j < R; j++) {
    available[j] -= req[j];
    alloc[p][j] += req[j];
    need[p][j] -= req[j];
}

// Cek apakah aman
if (isSafe()) {
    printf("Request disetujui. Sistem dalam keadaan aman.\n");
    return 1;
} else {
    // Kembalikan
    for (int j = 0; j < R; j++) {
        available[j] += req[j];
        alloc[p][j] -= req[j];
        need[p][j] += req[j];
    }
    printf("Request ditolak! Sistem menjadi tidak aman.\n");
    return 0;
}
}

```

```

int main() {

    printf("=== Banker's Algorithm ===\n");

    // Input Max matrix
    printf("\nMasukkan matriks MAX (%d proses × %d resource):\n", P, R);
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            scanf("%d", &max[i][j]);

    // Input Allocation matrix
    printf("\nMasukkan matriks ALLOCATION (%d proses × %d resource):\n", P, R);
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            scanf("%d", &alloc[i][j]);

    // Hitung Need = Max - Allocation
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    int choice;
    while (1) {
        printf("\nMenu:\n1. Request Resource\n2. Cek Safe State\n3. Exit\nPilihan: ");
        scanf("%d", &choice);

        if (choice == 1) {
            int p, req[R];
            printf("Masukkan nomor proses (0-%d): ", P-1);
            scanf("%d", &p);

```

```

printf("Masukkan request %d resource: ", R);
for (int j = 0; j < R; j++)
    scanf("%d", &req[j]);

requestResource(p, req);
}
else if (choice == 2) {
    if (isSafe())
        printf("Sistem dalam keadaan AMAN.\n");
    else
        printf("Sistem TIDAK AMAN.\n");
}
else if (choice == 3) {
    break;
}
else {
    printf("Pilihan tidak valid!\n");
}
}

return 0;
}

```

Ouput :

```
root@DESKTOP-J9C60BQ:~# ./bankers_4proses
=== Banker's Algorithm ===

Masukkan matriks MAX (4 proses x 3 resource):
7 5 3
3 2 2
9 0 2
2 2 2

Masukkan matriks ALLOCATION (4 proses x 3 resource):
0 1 0
2 0 0
3 0 2
1 0 1

Menu:
1. Request Resource
2. Cek Safe State
3. Exit
Pilihan: 2
Sistem dalam keadaan AMAN.

Menu:
1. Request Resource
2. Cek Safe State
3. Exit
Pilihan: 1
Masukkan nomor proses (0-3): 1
Masukkan request 3 resource: 1 0 2
Request disetujui. Sistem dalam keadaan aman.

Menu:
1. Request Resource
2. Cek Safe State
3. Exit
Pilihan: 2
Sistem dalam keadaan AMAN.

Menu:
1. Request Resource
2. Cek Safe State
3. Exit
Pilihan: 1
Masukkan nomor proses (0-3): 0
Masukkan request 3 resource: 5 2 3
Resource tidak cukup. Proses harus menunggu.
```


KASUS NYATA (REAL CASE)

```
#include <stdio.h>

#include <pthread.h>

#include <unistd.h>

pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;

void lock_ordered(pthread_mutex_t* a, pthread_mutex_t* b) {
    if (a < b) {
        pthread_mutex_lock(a);
        pthread_mutex_lock(b);
    } else {
        pthread_mutex_lock(b);
        pthread_mutex_lock(a);
    }
}

void unlock_ordered(pthread_mutex_t* a, pthread_mutex_t* b) {
    pthread_mutex_unlock(a);
    pthread_mutex_unlock(b);
}

void* transfer_A() {
    lock_ordered(&rekening1, &rekening2);
    printf("A: Transfer 1 -> 2\n");
    sleep(1);
    unlock_ordered(&rekening1, &rekening2);
    return NULL;
}
```

```

void* transfer_B() {
    lock_ordered(&rekening1, &rekening2);
    printf("B: Transfer 2 -> 1\n");
    sleep(1);
    unlock_ordered(&rekening1, &rekening2);
    return NULL;
}

int main() {
    pthread_t A, B;

    printf("=== SIMULASI TANPA DEADLOCK (SAFE) ===\n\n");

    pthread_create(&A, NULL, transfer_A, NULL);
    pthread_create(&B, NULL, transfer_B, NULL);

    pthread_join(A, NULL);
    pthread_join(B, NULL);

    printf("\nSemua transfer selesai tanpa deadlock.\n");
    return 0;
}

```

Output :

```

root@DESKTOP-J9C60BQ:~# ./bankers_transfer
=== SIMULASI TANPA DEADLOCK (SAFE) ===

A: Transfer 1 -> 2
B: Transfer 2 -> 1

Semua transfer selesai tanpa deadlock.

```

KESIMPULAN

Deadlock merupakan salah satu permasalahan kritis dalam sistem operasi maupun pemrograman paralel yang terjadi ketika dua atau lebih proses saling menunggu resource yang sedang dipegang satu sama lain, sehingga tidak ada proses yang dapat melanjutkan eksekusi. Deadlock muncul akibat adanya ketergantungan antar proses dalam penggunaan resource, khususnya ketika resource tersebut bersifat eksklusif dan tidak memungkinkan digunakan secara bersamaan. Bila tidak ditangani, deadlock dapat menyebabkan program berhenti merespons, kinerja sistem menurun, bahkan berpotensi menyebabkan kehilangan data.

SARAN

1. Gunakan urutan penguncian (lock ordering) yang konsisten ketika memerlukan lebih dari satu resource. Dengan memastikan bahwa seluruh thread menggunakan urutan lock yang sama, potensi circular wait dapat dihilangkan sepenuhnya.
2. Gunakan mekanisme try-lock untuk menghindari proses menunggu tanpa batas waktu. Jika pengambilan lock gagal, lepaskan resource yang sudah dipegang dan lakukan retry dengan jeda waktu yang sesuai.
3. Terapkan timeout pada operasi locking sehingga thread tidak menunggu selamanya ketika terjadi kompetisi resource yang tinggi.
4. Gunakan algoritma pencegahan deadlock, seperti Banker's Algorithm, pada sistem yang memiliki banyak proses dan resource, terutama pada sistem perbankan, transaksi database, dan manajemen memori.
5. Lakukan analisis dependensi resource sejak tahap desain, sehingga pola penguncian yang berpotensi menghasilkan siklus dapat segera dimodifikasi sebelum program dikembangkan lebih jauh.

